

Este archivo `.htaccess` configura reglas de reescritura de URLs utilizando el módulo `mod_rewrite` de Apache.

1. `RewriteEngine On`

- Activa el motor de reescritura de Apache.
 - Es necesario para que las siguientes reglas de reescritura funcionen.
-

2. `RewriteCond %{REQUEST_FILENAME} !-f`

- `RewriteCond` define una condición que debe cumplirse antes de aplicar la regla (`RewriteRule`).
- `%{REQUEST_FILENAME}` representa la ruta completa del archivo solicitado en el servidor.
- `!-f` significa "si no es un archivo físico".
 - Es decir, esta condición se cumple si el archivo solicitado **no existe** físicamente en el servidor.

Ejemplo:

- Si el cliente solicita `example`, y el archivo `example` **no** existe, esta condición se cumple.
 - Si el cliente solicita `example.php` y ese archivo **sí** existe, esta condición **no se cumple** y la regla no se aplica.
-

3. `RewriteRule ^([^\.]*)$ $1.php [NC,L]`

Esta línea define la regla de reescritura:

a) `^([^\.]*)$`

- Este es un patrón de expresión regular que coincide con la URL solicitada:
 - `^`: Indica el inicio de la cadena.
 - `([^\.]*)`: Captura cualquier conjunto de caracteres que **no incluya un punto (.)**.
 - El grupo capturado (entre paréntesis) se almacena como `$1`.
 - `$`: Indica el final de la cadena.
- En resumen, el patrón coincide con cualquier solicitud de URL que no contenga un punto (.), como `example` o `contact`, pero no `example.php` ni `style.css`.

b) `$1.php`

- Esto indica la URL de destino que Apache debe procesar.

- **\$1**: Se reemplaza con el valor capturado del patrón.
- **.php**: Se agrega la extensión `.php` al final.

Ejemplo:

- Si el cliente solicita `example`, Apache redirigirá internamente a `example.php`.
- Si solicita `contact`, Apache redirigirá internamente a `contact.php`.

c) [NC,L]

Estas son las banderas que modifican el comportamiento de la regla:

- **NC** (No Case): Hace que la comparación sea insensible a mayúsculas y minúsculas.
 - Por ejemplo, tanto `Example` como `example` coincidirán con el patrón.
- **L** (Last): Indica que esta es la última regla que se debe aplicar si coincide.
 - Evita que otras reglas posteriores sean evaluadas.

¿Qué hace este `.htaccess` en conjunto?

1. Activa el motor de reescritura (`RewriteEngine On`).
2. Comprueba si el archivo solicitado **no** existe físicamente (`RewriteCond %{REQUEST_FILENAME} !-f`).
3. Si el archivo no existe y la URL **no contiene un punto** (`.`), reescribe la URL para agregar la extensión `.php`.

Ejemplos prácticos

Supongamos que este `.htaccess` está en el directorio raíz del sitio:

1. El cliente solicita `http://tu-sitio.com/contact`:
 - Apache comprueba si existe un archivo llamado `contact`. Como no lo encuentra, redirige internamente a `contact.php`.
 2. El cliente solicita `http://tu-sitio.com/about`:
 - Se redirige internamente a `about.php`.
 3. El cliente solicita `http://tu-sitio.com/style.css`:
 - No se aplica la regla porque la URL contiene un punto (`.`).
-

Ventajas de este enfoque

- **URLs limpias y amigables:** Permite ocultar la extensión `.php` de las URLs.
 - **Flexibilidad:** Las URLs son más fáciles de recordar para los usuarios.
 - **No afecta archivos estáticos:** La condición `RewriteCond %{REQUEST_FILENAME} !-f` asegura que archivos como imágenes, CSS o JavaScript no sean reescritos.
-

Consideraciones

- Asegúrate de que los archivos `.php` existan, ya que las solicitudes se redirigirán a ellos.
- Si se espera que ciertos archivos (como `robots.txt`) sean accesibles, deben existir físicamente para evitar errores 404.